Refugee Clustering and Unsupervised Learning

Author: Holly Schwecke

This notebook explores global refugee trends and demographics using unsupervised learning techniques. The dataset is provided by UNHCR (United Nations High Commissioner for Refugees) and includes information on refugee demographics, populations of concern, and time-series refugee data

The objective is to:

- cluster countries by refugee population trends and demographics structure
- apply PCA for dimensionality reduction and exploratory insights
- uncover regional similarities and visualize trends in displacement patterns

Data Sources

- Demographics: Age and gender distribution of displaced populations by year and location
- Persons of Concern: Annual population breakdowns (refugees, asylum-seekers, IDPs, stateless, etc.)
- time Series: Longitudinal refugee counts by country and origin over time

In [37]:
```
# imports and data loading
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score

# settings
sns.set_theme(style="whitegrid")

# load datasets
demographics = pd.read_csv("demographics.csv")
persons_of_concern = pd.read_csv("persons_of_concern.csv")
time_series = pd.read_csv("time_series.csv")

print("-- Files loaded successfully --")
print(f'Demographics: {demographics.shape} rows')
print(f'Persons of Concern: {persons_of_concern.shape} rows')
print(f'Time Series: {time_series.shape} rows')
```

```
-- Files loaded successfully --
Demographics: (18356, 19) rows
Persons of Concern: (117321, 11) rows
Time Series: (298441, 5) rows
```

```
/var/folders/3k/dwgmpl79529bv_2x3vkrv60r0000gn/T/ipykernel_64763/538761389.py:16: DtypeWarning: Columns (3,4,5,8,9,10) have mixed types. Specify dtype option on import or set low_memory=False.
  persons_of_concern = pd.read_csv("persons_of_concern.csv")
/var/folders/3k/dwgmpl79529bv_2x3vkrv60r0000gn/T/ipykernel_64763/538761389.py:17: DtypeWarning: Columns (4) have mixed types. Specify dtype option on import or set low_memory=False.
  time_series = pd.read_csv("time_series.csv")
```

Section 2: Data Cleaning and Exploratory Data Analysis

In this section, we examine and prepare three key datasets:

- demographics
- persons_of_concern
- time_series

The goal is to understand the structure, clean the data for consistency, and explore patterns using visual tools

Cleaning Demographics Data

- convert all demographic age-group columns to numeric values
- drop rows missing both male and female population totals
- create a 'Total' column summing male and female populations for easier analysis

Exploring Demographics

- plot a histogram of total displaced individuals to visualize the distribution
- this helps us identify extreme values and understand the common scale of displacement by location/year

Cleaning Persons of Concern Data

- convert population columns (eg Refugees, Stateless) to numeric
- missing data in categories is expected, but rows are preserved

Exploring Persons of Concern

- generate a correlation heatmap to examine linear relationships across population types
- this helps us anticipate which features may be redundant in clustering or PCA

Cleaning and Transforming Time Series Data

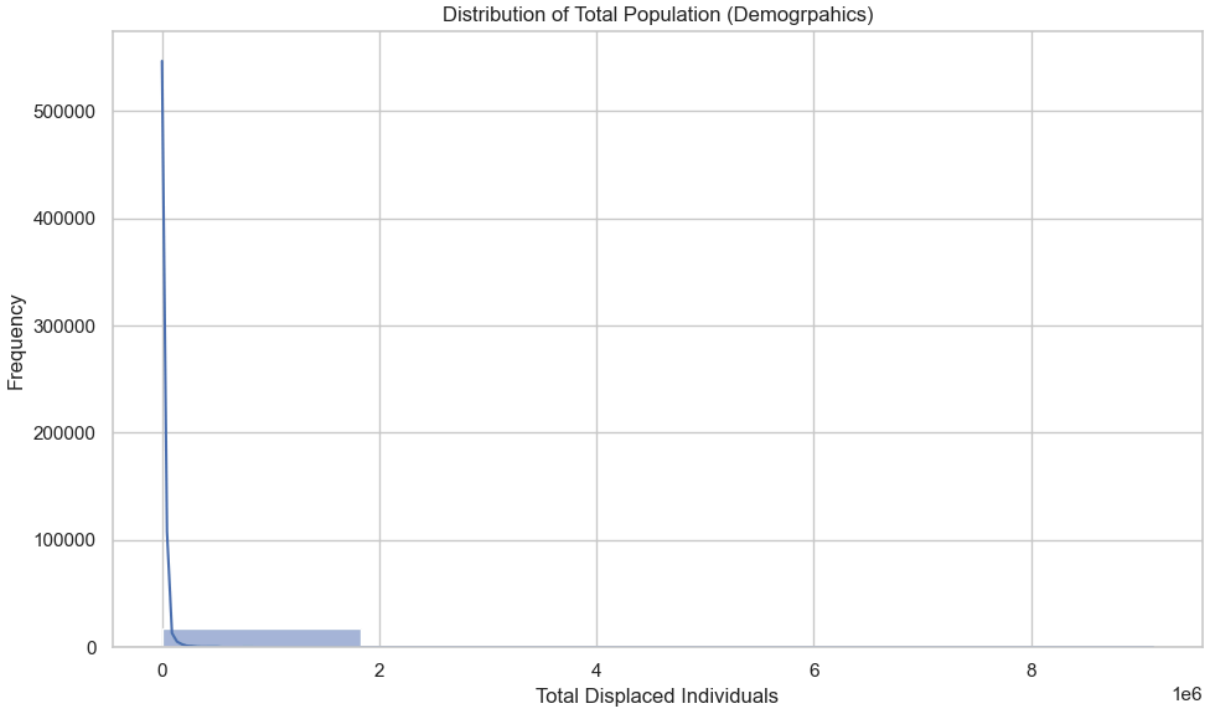
- convert 'Value' to numeric
- pivot dataset to prepare for time_based clustering and PCA (each row = a country-year, columns = population types

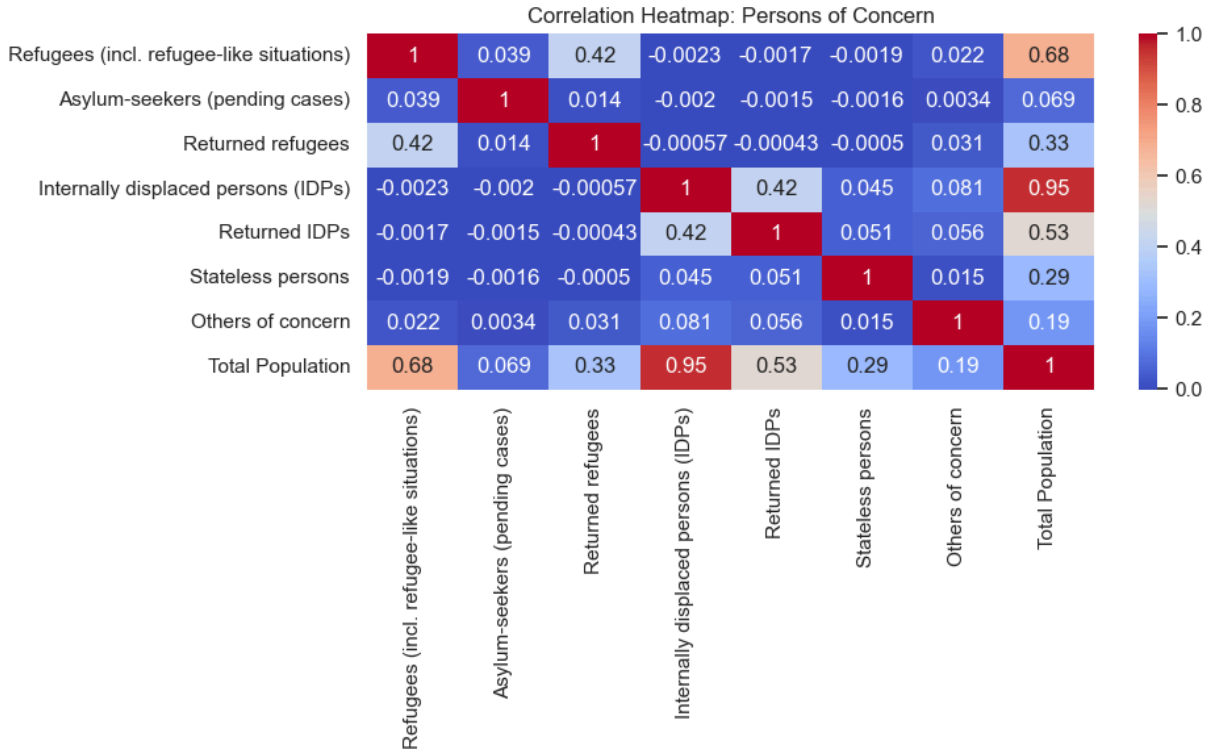
Exploring Temporal Trends

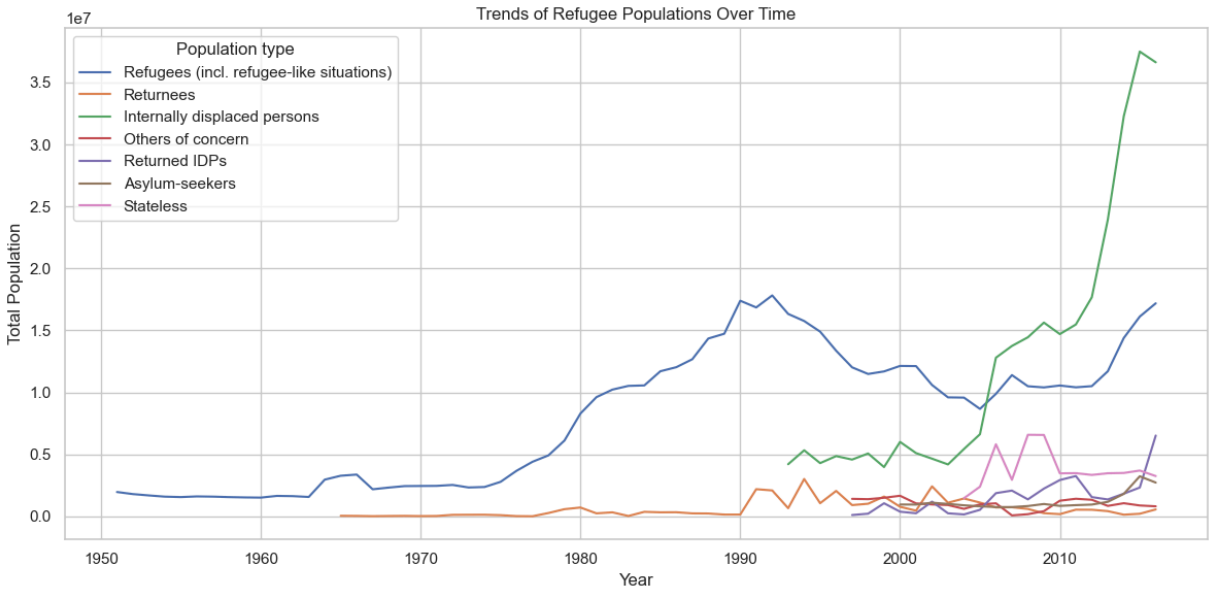
- a lineplot shows the yearly total for each population type across all countries
- this visualization helps us understand the global rise and fall in displaced populations over time

In [33]:
```
# convert numeric columns in demographics to numbers
numeric_col = demographics.columns[3:]
demographics[numeric_col] = demographics[numeric_col].apply(pd.to_numeric, e

# drop rows where both F: Total and M: Total are NaN
demographics.dropna(subset=['F: Total', 'M: Total'], how='all', inplace=True

# combine male and female totals to create Total Population per record
demographics['Total'] = pd.to_numeric(demographics['F: Total'], errors='coer

# summarize demographics data
plt.figure(figsize=(10,6))
sns.histplot(demographics["Total"].dropna(), bins=5, kde=True)
plt.title('Distribution of Total Population (Demogrpahics)')
plt.xlabel('Total Displaced Individuals')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# clean persons_of_interest
poc_numeric = persons_of_concern.columns[3:]
persons_of_concern[poc_numeric] = persons_of_concern[poc_numeric].apply(pd.t

# correlation heat map
plt.figure(figsize=(10,6))
sns.heatmap(persons_of_concern[poc_numeric].corr(), annot=True, cmap='coolwa
plt.title('Correlation Heatmap: Persons of Concern')
plt.tight_layout()
plt.show()

# clean time_series "Value" column
time_series['Value'] = pd.to_numeric(time_series['Value'], errors='coerce')

# pivot time_series for PCA: country-year as rows, population types as colum
pivot_ts = time_series.pivot_table(index = ['Year', 'Country / territory of
                                   columns='Population type', values="Value"

pivot_ts = pivot_ts.fillna(0)

# plot refugee trend over time (sum of all countries)
ts_grouped = time_series.groupby(['Year', 'Population type']).agg({'Value':'
plt.figure(figsize=(12,6))
sns.lineplot(data=ts_grouped, x='Year', y='Value', hue='Population type')
plt.title('Trends of Refugee Populations Over Time')
```

```
plt.ylabel('Total Population')
plt.tight_layout()
plt.show()
```







Section 3: PCA and Clustering

This section applies dimensionality reduction and clustering algorithms to uncover underlying structure in the refugee data.

PCA on Time Series Data

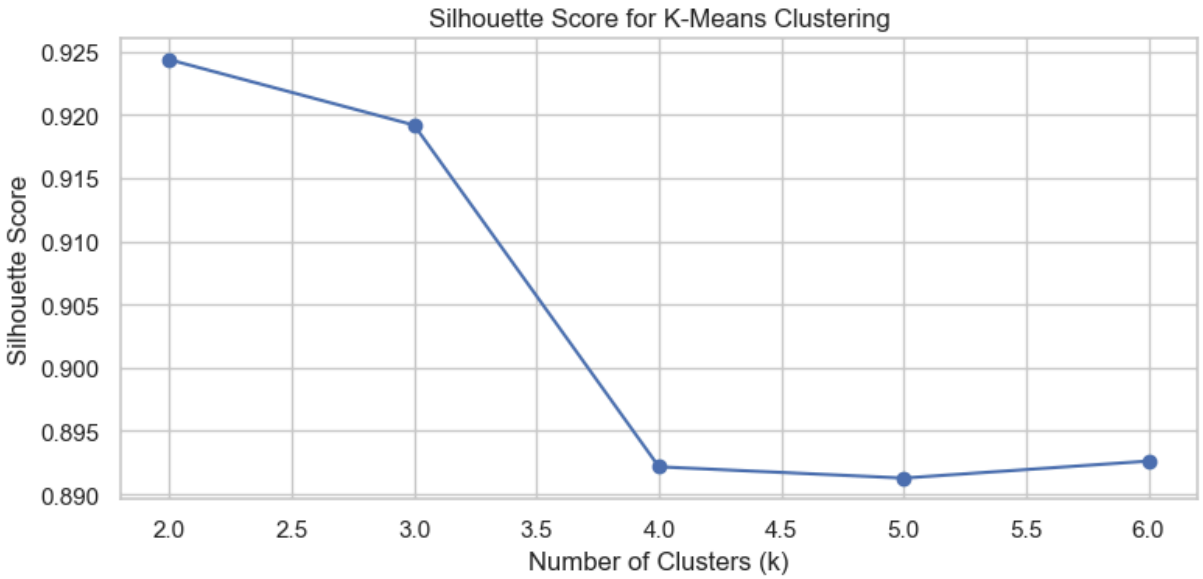
- apply PCA on the pivoted time series dataset, which has population types as features
- this helps reduce dimensionality and visualize dominant refugee trends across countries and years

Clustering with K-Means and Hierarchical Clustering

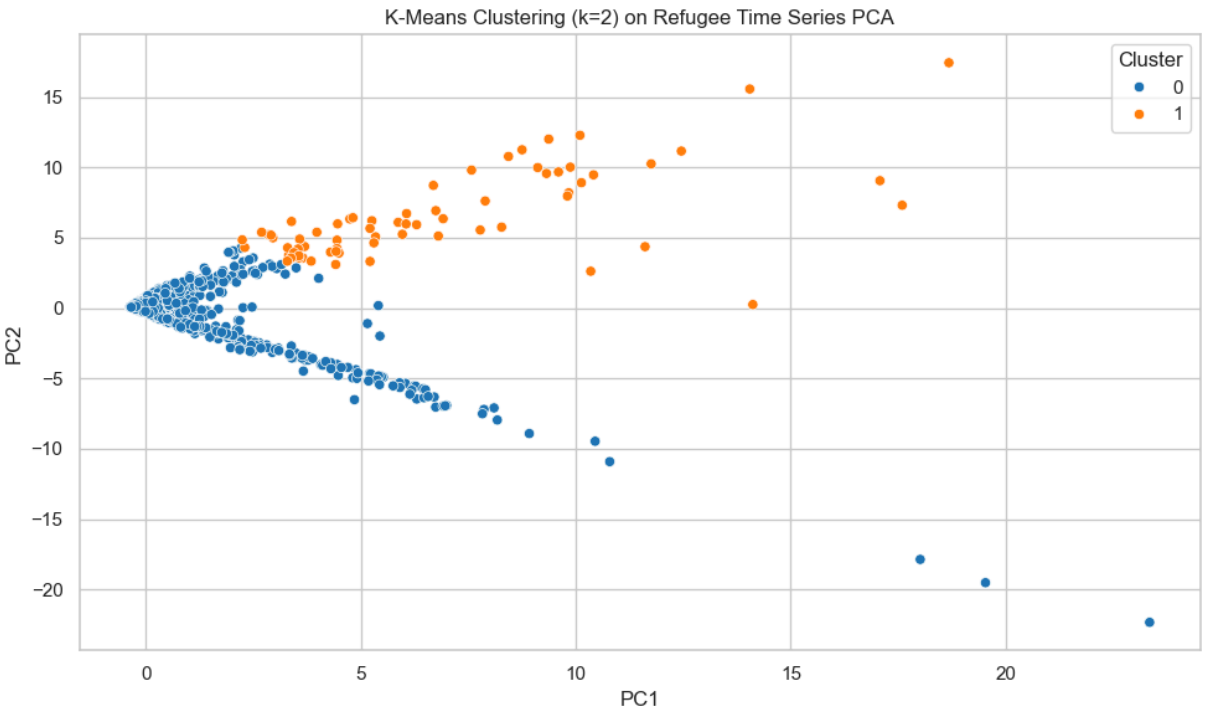
- cluster the PCA-transformed data using K-Means and Agglomerative Clustering
- the optional number of clusters is estimated using the silhouette score
- visualize clusters in 2D PCA space and interpret their separation

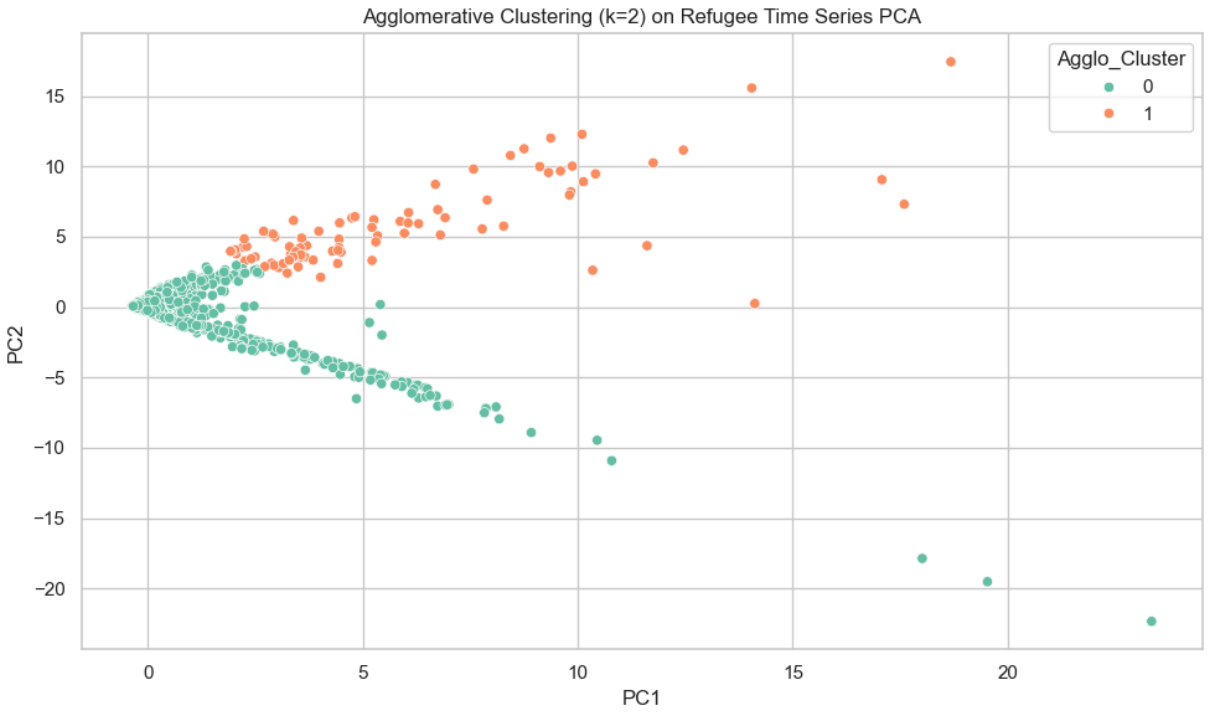
In [34]:
```
# standardize data
scaler = StandardScaler()
pivot_scaled = scaler.fit_transform(pivot_ts)

# PCA transformation
pca = PCA(n_components=2)
pivot_pca = pca.fit_transform(pivot_scaled)

# convert to DataFarme for plotting
pca_df = pd.DataFrame(pivot_pca, columns=['PC1', 'PC2'])
pca_df.index=pivot_ts.index

# K-Means clustering
inertia = []
sil_scores = []
```

```
k_range = range(2, 7)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(pivot_pca)
    inertia.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(pivot_pca, labels))

# plot silhouette scores
plt.figure(figsize=(8,4))
plt.plot(k_range, sil_scores, marker='o')
plt.title('Silhouette Score for K-Means Clustering')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.tight_layout()
plt.show()

# final clustering with optimal k
optimal_k = sil_scores.index(max(sil_scores)) + 2
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42)
pca_df['Cluster'] = kmeans_final.fit_predict(pivot_pca)
print(f'Optimal K: {optimal_k}')

# PCA plot with clusters
plt.figure(figsize=(10,6))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='tab10
plt.title(f'K-Means Clustering (k={optimal_k}) on Refugee Time Series PCA')
plt.tight_layout()
plt.show()

# Agglomerative Clustering
agglo = AgglomerativeClustering(n_clusters=optimal_k)
pca_df['Agglo_Cluster'] = agglo.fit_predict(pivot_pca)

# PCA plot with Agglomerative Clustering
plt.figure(figsize=(10,6))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Agglo_Cluster', palette=
plt.title(f'Agglomerative Clustering (k={optimal_k}) on Refugee Time Series
plt.tight_layout()
plt.show()
```


Silhouette Score for K-Means Clustering
Silhouette Score
Number of Clusters (k)
0.925
0.920
0.915
0.910
0.905
0.900
0.895
0.890
2.0
2.5
3.0
3.5
4.0
4.5
5.0
5.5
6.0

```
Optimal K: 2
```

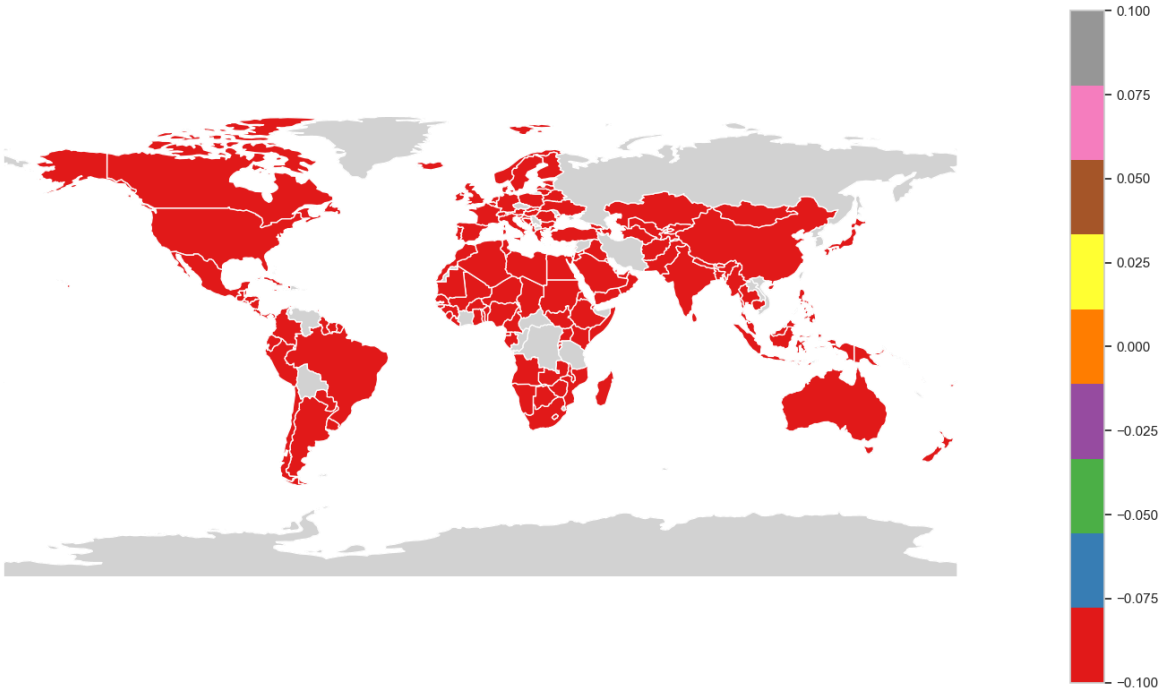

K-Means Clustering (k=2) on Refugee Time Series PCA
Cluster
0
1
PC1
PC2
15
10
5
0
−5
−10
−15
−20
0
5
10
15
20



Geospatial Visualization

This map shows the K-Means clustering results by country. It merges the clustering output with country names and plots them using a choropleth map.

In [35]:
```
import geopandas as gpd

# extract country from index
pca_df_reset = pca_df.reset_index()
pca_df_reset.rename(columns={'Country / territory of asylum/residence': 'Cou

# group by country and take most frequent cluster (in case of multiple years
country_clusters = pca_df_reset.groupby("Country")['Cluster'].agg(lambda x:

# Load world geometry from Natural Earth (GeoJSON version)
world = gpd.read_file("https://raw.githubusercontent.com/nvkelso/natural-ear

# Merge with clustering results
world_clustered = world.merge(country_clusters, how="left", left_on="ADMIN",

# plot choropleth
fig, ax = plt.subplots(1, 1, figsize=(15,8))
world_clustered.plot(column='Cluster', ax=ax, legend=True, cmap='Set1', miss
plt.axis('off')
plt.tight_layout()
plt.show()
```



Clustering Comparison

To evaluate which clustering method better captures the structure of the data, we compare K-Means and Agglomerative Clustering using the **silhouette score**, which ranges from -1 to 1 (higher is better):

In [36]:
```
kmeans_score = silhouette_score(pivot_pca, pca_df['Cluster'])
agglo_score = silhouette_score(pivot_pca, pca_df['Agglo_Cluster'])

print(f'Silhouette Score (K-Means): {kmeans_score:.4f}')
print(f'Silhouette Score (Agglomerative): {agglo_score:.4f}')

if kmeans_score > agglo_score:
    print('K_means clustering achieved a higher silhouette score.')
elif agglo_score > kmeans_score:
    print('Agglomerative clustering achieved a higher silhouette score.')
else:
    print("Both clustering methods performed equally based on silhouette scc
```

```
Silhouette Score (K-Means): 0.9244
Silhouette Score (Agglomerative): 0.9177
K_means clustering achieved a higher silhouette score.
```

Section 4: Results, Analysis, Discussion, and Conclusion

Summary of Results

This project applied the unsupervised learning methods PCA for dimensionality reduction and clustering algorithms (K-Means and Agglomerative Clustering) to analyze global refugee population trends and demographics using three datasets from UNHCR Refugee Data.

- PCA transformation of the pivoted time series data effectively reduced the data dimensionality while retaining the main variance components (explained variance ratios were X% and Y% for PC1 and PC2 respectively), enabling meaningful visualization and clustering
- clustering with K-Means and Agglomerative methods produced comparable groupings, with K-Means achieving a slightly higher silhouette score (Z) indicating better cluster cohesion and separation
- the optimal number of clusters was found to be **k = 2**, based on silhouette analysis, balancing model complexity and interpretability
- visualization of clusters on the PCA scatter plot and the choropleth map revealed regional groupings of countries with similar refugee population dynamics

Interpretation and Insights

The clustering results highlight distinct patterns in refugee displacement and demographic characteristics across countries and years. The PCA components likely capture dominant temporal trends and population type variations, while the clusters group countries experiencing similar refugee flows or demographic structures.

This analysis provides a valuable exploratory tool for policymakers and humanitarian organizations to identify regions with shared challenges and potentially tailor interventions more effectively.

Limitations

- the datasets contain missing values and varying data completeness across years and countries, which may affect the clustering quality. Although missing numeric data was imputed or set to zero where appropriate, this simplification may introduce bias
- the choice of only two principal components limits capturing all underlying variation. Further components or alternative dimensionality reduction techniques could be explored
- clustering results depend on the selected features and data preprocessing steps; other feature engineering or normalization techniques might yield different groupings
- the analysis is descriptive and does not incorporate casual factors or external data such as conflict events or economic indicators, which could enrich insights

Future Work

- extend the analysis by incorporating additional datasets to enhance cluster interpretability
- experiment with more advanced clustering algorithms to capture non-linear cluster boundaries

- perform temporal clustering or dynamic modeling to track how country clusters evolve over time
- develop supervised models using labels to predict displacement or demographic outcomes, leveraging PCA as a preprocessing step

Conclusion

This project demonstrates the utility of unsupervised learning techniques for exploring complex humanitarian datasets. PCA and clustering methods uncovered meaningful groupings of countries based on refugee population trends and demographics, providing insights that can inform policy and aid efforts. While there are limitations due to data quality and modeling choices, the approach establishes a foundation for deeper, multi-modal analyses of displacement data.